

Containerization of Flask Application using Podman on RHEL

Objective

The main objective of this experiment is to containerize a Flask-based web application using Podman on Red Hat Enterprise Linux (RHEL) and manage its source code using Git and GitHub.

This includes configuring Git for version control, creating and building a container image using a Dockerfile, running the Flask application inside a Podman container, and verifying its functionality through HTTP requests.

Additionally, the task demonstrates how to push the project to a remote GitHub repository, showcasing the integration of containerization and version control tools in a modern DevOps workflow.

Introduction

The objective of this practical experiment is to demonstrate how to containerize a Flask-based web application using Podman on Red Hat Enterprise Linux (RHEL) and manage its source code through Git and GitHub. Podman, being a daemonless container engine, provides a lightweight and secure alternative to Docker for building, running, and managing containers.

In this project, a simple Flask web application was created and packaged inside a Podman container. The experiment also included version control operations such as configuring Git, committing changes, and pushing the code to a remote GitHub repository. This exercise illustrates the complete DevOps workflow — from application creation, image building, and container deployment to Git-based collaboration and repository management — in a RHEL environment.

Commands and Explanations

Purpose:

This command lists all running containers on the system.

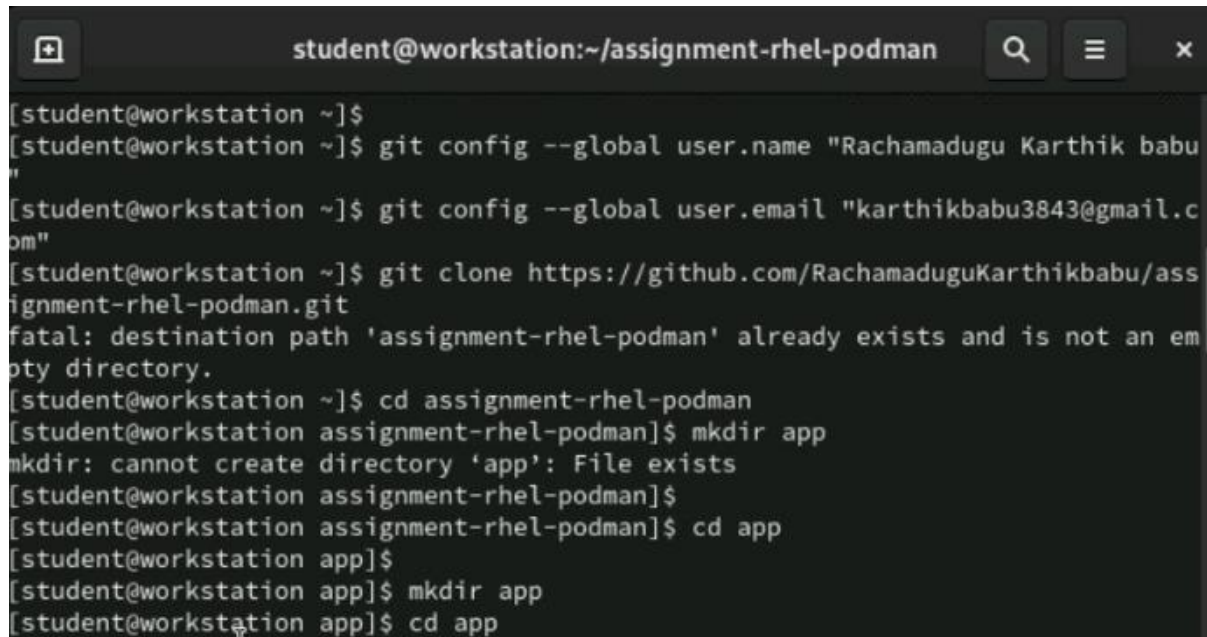
Explanation:

It shows details like container ID, image name, command used to start the container, status, ports, and container names. Initially, this command was used to verify if any containers were active before starting new ones. Since no containers were running, it displayed an empty result.

Git Configuration and Repository Setup

Purpose:

To configure Git with the user's name and email for commit tracking, clone the GitHub repository to the local system, and create a working directory for the Flask application.

A terminal window titled 'student@workstation:~/assignment-rhel-podman' with search, menu, and close icons. The terminal shows the following commands and output:

```
[student@workstation ~]$  
[student@workstation ~]$ git config --global user.name "Rachamadugu Karthik babu"  
[student@workstation ~]$ git config --global user.email "karthikbabu3843@gmail.com"  
[student@workstation ~]$ git clone https://github.com/RachamaduguKarthikbabu/assignment-rhel-podman.git  
fatal: destination path 'assignment-rhel-podman' already exists and is not an empty directory.  
[student@workstation ~]$ cd assignment-rhel-podman  
[student@workstation assignment-rhel-podman]$ mkdir app  
mkdir: cannot create directory 'app': File exists  
[student@workstation assignment-rhel-podman]$  
[student@workstation assignment-rhel-podman]$ cd app  
[student@workstation app]$  
[student@workstation app]$ mkdir app  
[student@workstation app]$ cd app
```

Explanation:

The git config commands set the username and email globally, which are attached to all commits made by the user.

git clone downloads the repository assignment-rhel-podman from GitHub into the local system for further development.

The mkdir app and cd app commands create and navigate into a new folder named app, which will store the Flask application files.

Creating a Flask Application File (app.py)

Purpose:

This command creates a new Python file named app.py containing the Flask web application code.

```
[student@workstation app]$ cat <<EOF > app.py
>
> from flask import Flask
>
> app = Flask(__name__)
>
>
>
> @app.route("/")
>
> def home():
>
>     return "Hello from Podman container on RHEL!"
>
>
>
> if __name__ == "__main__":
>
>     app.run(host="0.0.0.0", port=8080)
>
> EOF
[student@workstation app]$
```

Explanation:

The `cat <<EOF > app.py` command is used to create and write multiple lines into a file directly from the terminal.

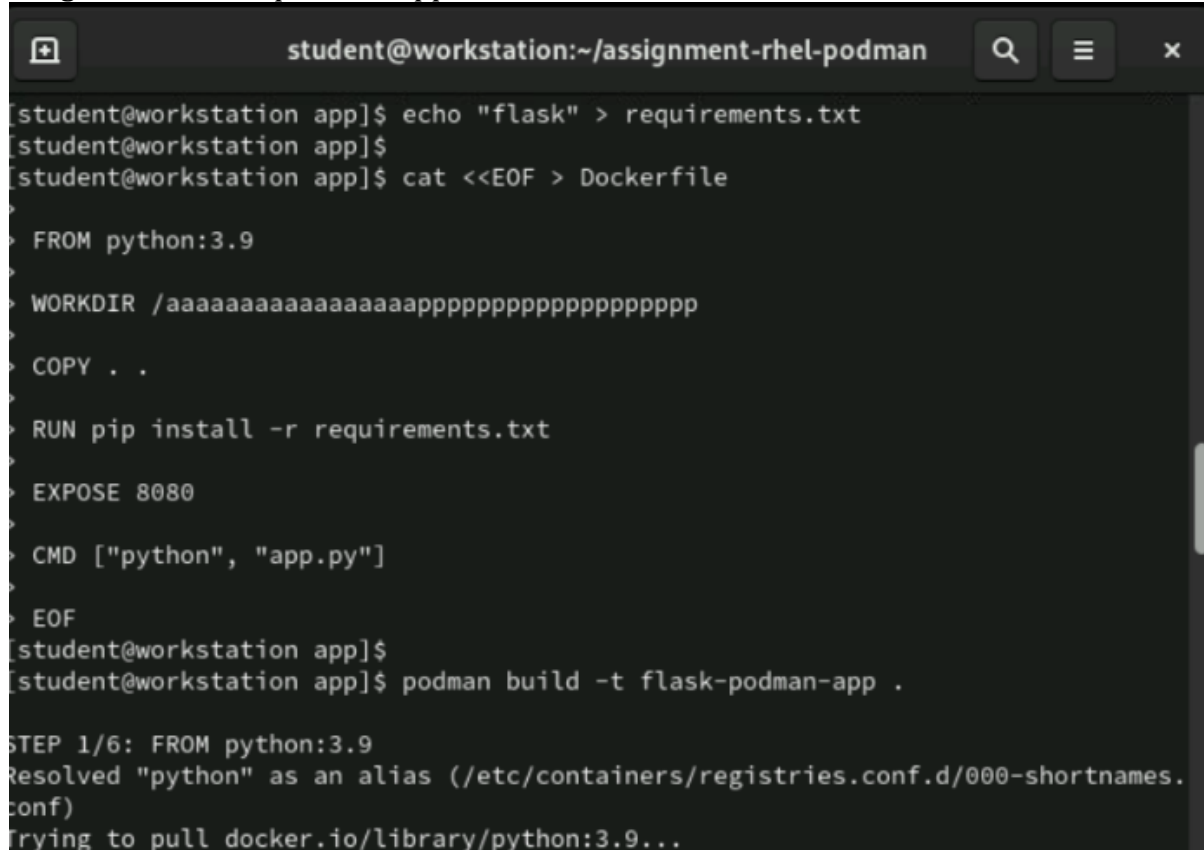
Inside the file, a Flask app is defined that displays the message *"Hello from Podman container on RHEL!"* when accessed via a web browser.

The `app.run()` function starts the web server on all interfaces (0.0.0.0) at port 8080, making it accessible from the container.

Creating Dockerfile and Building Flask Podman Image

Purpose:

To define the container build instructions for the Flask application and build a Podman image named flask-podman-app.

A terminal window titled 'student@workstation:~/assignment-rhel-podman' with search, menu, and close icons. The terminal shows the following commands and output:

```
student@workstation app]$ echo "flask" > requirements.txt
student@workstation app]$
student@workstation app]$ cat <<EOF > Dockerfile
FROM python:3.9
WORKDIR /aaaaaaaaaaaaaaaaapppppppppppppppppppppppp
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8080
CMD ["python", "app.py"]
EOF
student@workstation app]$
student@workstation app]$ podman build -t flask-podman-app .

STEP 1/6: FROM python:3.9
Resolved "python" as an alias (/etc/containers/registries.conf.d/000-shortnames.conf)
Trying to pull docker.io/library/python:3.9...
```

Explanation:

The command `echo "flask" > requirements.txt` creates a file listing Flask as the required dependency.

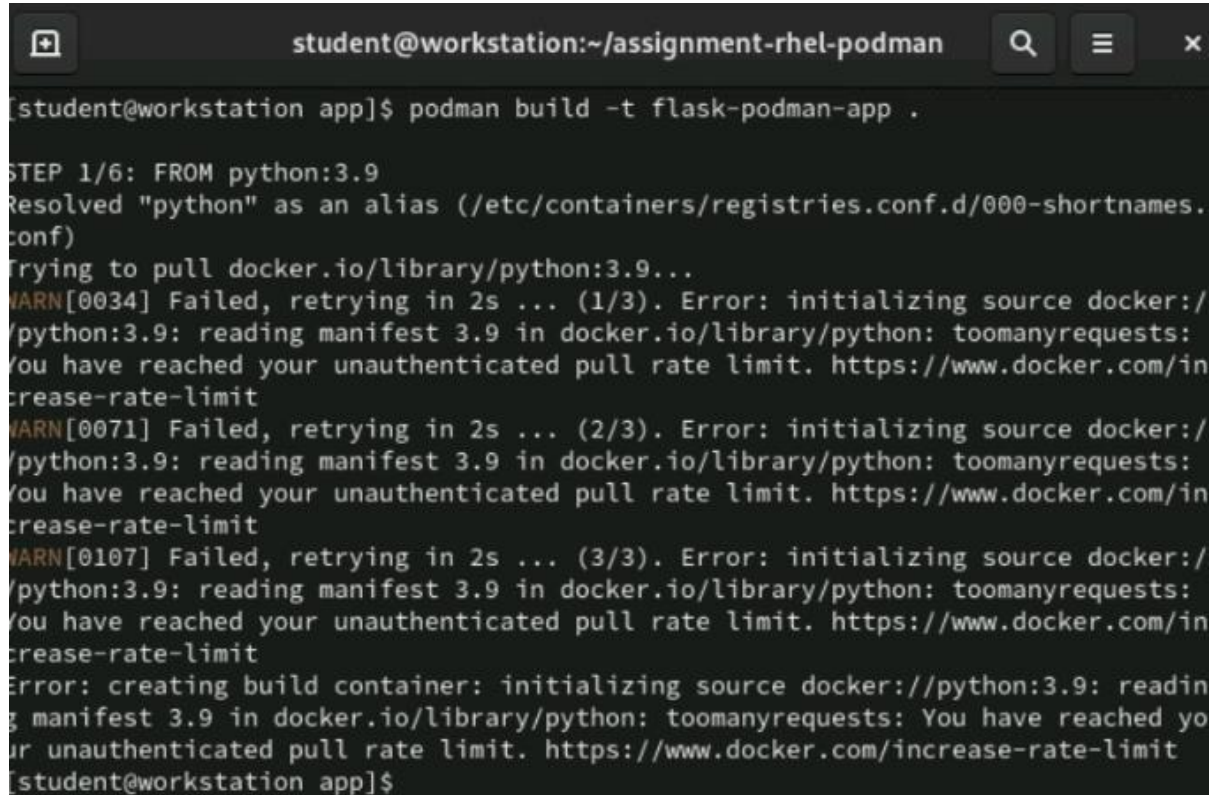
The Dockerfile uses the Python 3.9 base image, sets `/app` as the working directory, copies the application files, installs dependencies, and exposes port 8080.

Finally, `podman build -t flask-podman-app .` builds a container image tagged as flask-podman-app using the Dockerfile in the current directory.

Handling Podman Image Build Error (Docker Hub Rate Limit)

Purpose:

To build a container image named flask-podman-app using the Flask application's Dockerfile.

A terminal window titled 'student@workstation:~/assignment-rhel-podman' showing a Podman build command. The command is 'podman build -t flask-podman-app .' and it fails at STEP 1/6: FROM python:3.9. The error message is 'Error: creating build container: initializing source docker://python:3.9: reading manifest 3.9 in docker.io/library/python: toomanyrequests: You have reached your unauthenticated pull rate limit. https://www.docker.com/increase-rate-limit'. This message is repeated three times with warning codes [0034], [0071], and [0107].

```
student@workstation:~/assignment-rhel-podman
[student@workstation app]$ podman build -t flask-podman-app .

STEP 1/6: FROM python:3.9
Resolved "python" as an alias (/etc/containers/registries.conf.d/000-shortnames.conf)
Trying to pull docker.io/library/python:3.9...
WARN[0034] Failed, retrying in 2s ... (1/3). Error: initializing source docker://python:3.9: reading manifest 3.9 in docker.io/library/python: toomanyrequests: You have reached your unauthenticated pull rate limit. https://www.docker.com/increase-rate-limit
WARN[0071] Failed, retrying in 2s ... (2/3). Error: initializing source docker://python:3.9: reading manifest 3.9 in docker.io/library/python: toomanyrequests: You have reached your unauthenticated pull rate limit. https://www.docker.com/increase-rate-limit
WARN[0107] Failed, retrying in 2s ... (3/3). Error: initializing source docker://python:3.9: reading manifest 3.9 in docker.io/library/python: toomanyrequests: You have reached your unauthenticated pull rate limit. https://www.docker.com/increase-rate-limit
Error: creating build container: initializing source docker://python:3.9: reading manifest 3.9 in docker.io/library/python: toomanyrequests: You have reached your unauthenticated pull rate limit. https://www.docker.com/increase-rate-limit
[student@workstation app]$
```

Explanation:

During the build process, Podman attempted to pull the base image python:3.9 from Docker Hub but failed due to unauthenticated pull rate limits.

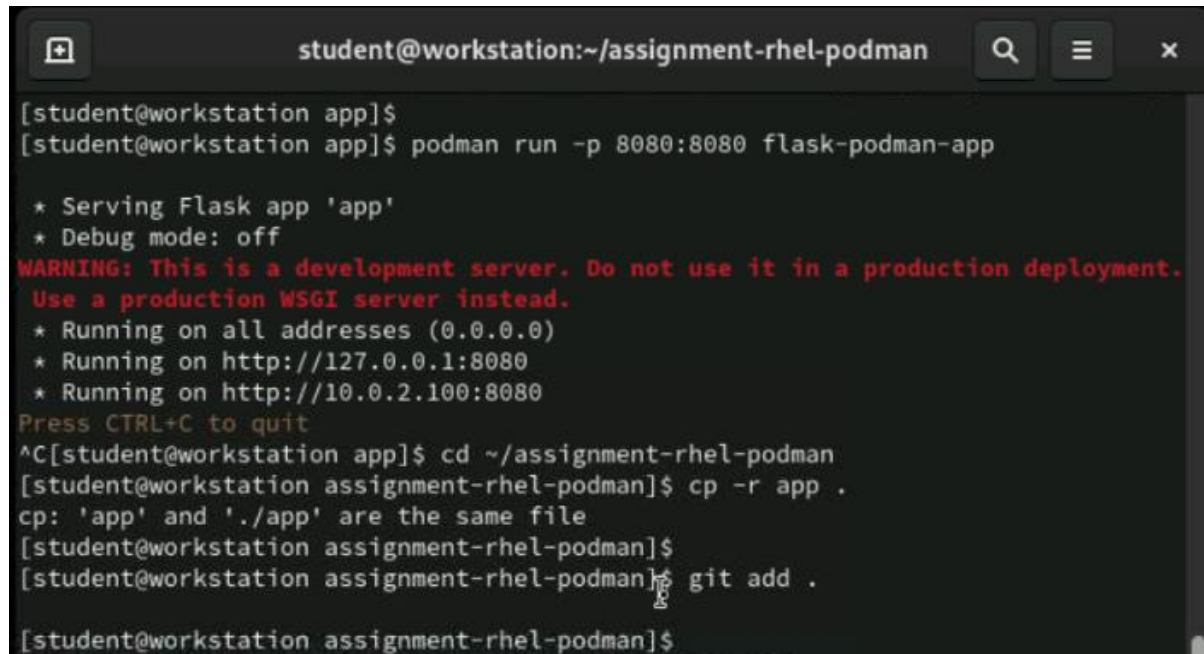
The warning messages (toomanyrequests) indicate that the maximum number of image pulls for unauthenticated users was reached.

This issue can be resolved by authenticating to Docker Hub using the podman login docker.io command, which allows additional pulls under a verified user account.

Running Flask Application Container and Adding Files to Git

Purpose:

To run the Flask container on port 8080 and prepare project files for version control using Git.



```
student@workstation:~/assignment-rhel-podman
[student@workstation app]$
[student@workstation app]$ podman run -p 8080:8080 flask-podman-app

* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://10.0.2.100:8080
Press CTRL+C to quit
^C[student@workstation app]$ cd ~/assignment-rhel-podman
[student@workstation assignment-rhel-podman]$ cp -r app .
cp: 'app' and './app' are the same file
[student@workstation assignment-rhel-podman]$
[student@workstation assignment-rhel-podman]$ git add .
[student@workstation assignment-rhel-podman]$
```

Explanation:

The `podman run -p 8080:8080 flask-podman-app` command starts the containerized Flask app and maps container port 8080 to the host system's port 8080, allowing external access.

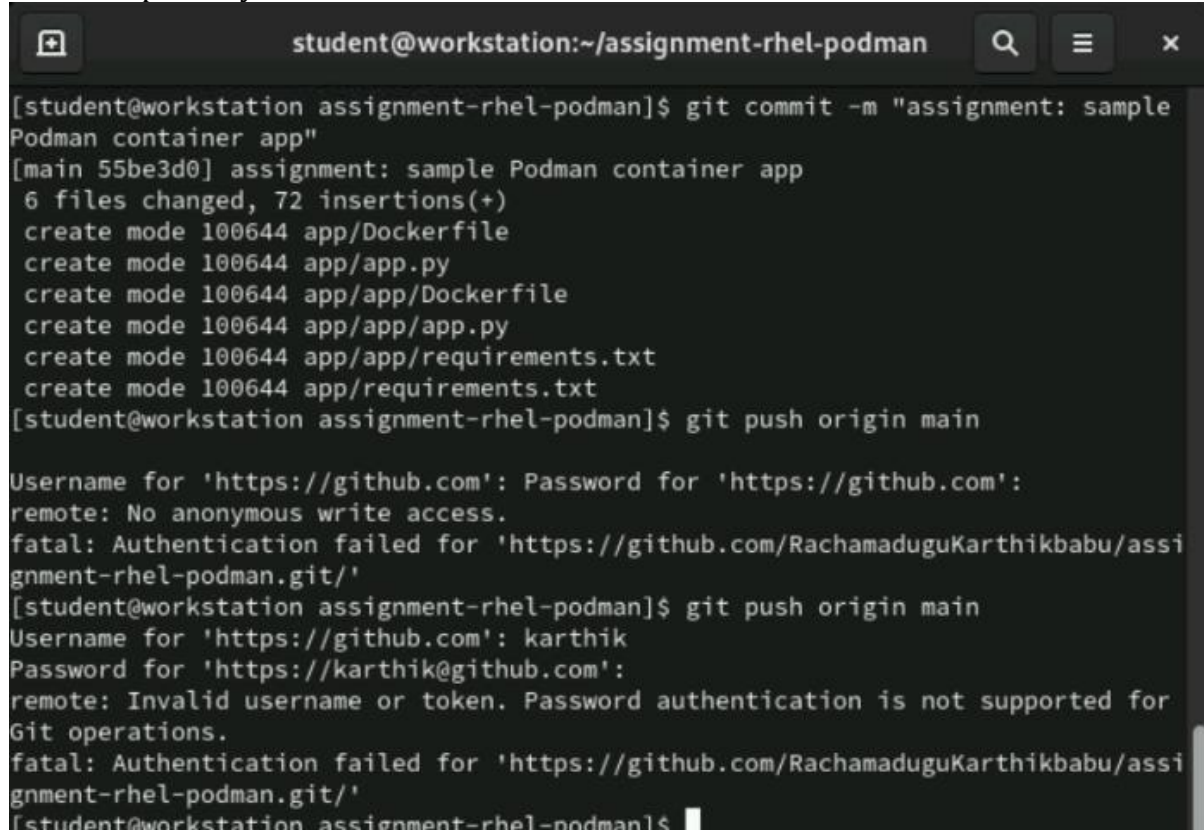
The output confirms the app is running successfully on `http://127.0.0.1:8080` and `http://10.0.2.100:8080`.

After testing, the user navigated back to the repository directory, copied the app folder, and used `git add .` to stage all files for commit.

Committing and Pushing Flask Podman Application to GitHub

Purpose:

To commit all staged project files to the local repository and push them to the remote GitHub repository.

A terminal window titled 'student@workstation:~/assignment-rhel-podman' showing the execution of git commands. The first command is 'git commit -m "assignment: sample Podman container app"', which successfully commits 6 files. The second command is 'git push origin main', which fails due to authentication issues. The terminal shows the prompt for a username and password, followed by error messages indicating that password authentication is not supported and that the username or token is invalid.

```
[student@workstation assignment-rhel-podman]$ git commit -m "assignment: sample Podman container app"
[main 55be3d0] assignment: sample Podman container app
6 files changed, 72 insertions(+)
create mode 100644 app/Dockerfile
create mode 100644 app/app.py
create mode 100644 app/app/Dockerfile
create mode 100644 app/app/app.py
create mode 100644 app/app/requirements.txt
create mode 100644 app/requirements.txt
[student@workstation assignment-rhel-podman]$ git push origin main

Username for 'https://github.com': Password for 'https://github.com':
remote: No anonymous write access.
fatal: Authentication failed for 'https://github.com/RachamaduguKarthikbabu/assignment-rhel-podman.git/'
[student@workstation assignment-rhel-podman]$ git push origin main
Username for 'https://github.com': karthik
Password for 'https://karthik@github.com':
remote: Invalid username or token. Password authentication is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/RachamaduguKarthikbabu/assignment-rhel-podman.git/'
[student@workstation assignment-rhel-podman]$
```

Explanation:

The git commit command records changes in the local repository with a descriptive message for tracking updates.

The git push origin main command attempts to upload these commits to the remote GitHub repository.

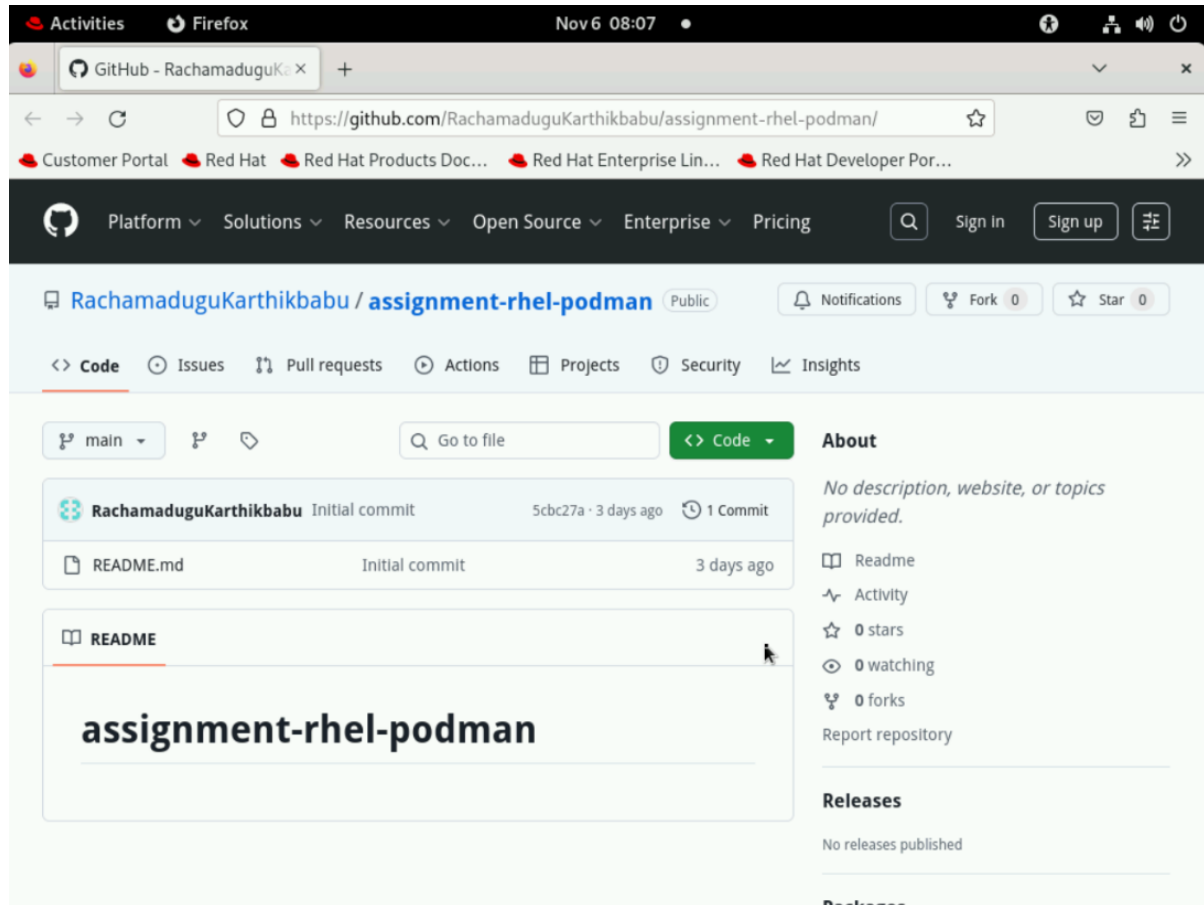
However, authentication failed because GitHub no longer supports password-based authentication.

To resolve this issue, users must use a Personal Access Token (PAT) instead of a password when performing Git operations over HTTPS.

Verifying Repository on GitHub

Purpose:

To verify that the repository and committed files have been successfully uploaded to GitHub.



Explanation:

After pushing the project, the GitHub repository page <https://github.com/RachamaduguKarthikbabu/assignment-rhel-podman> was opened in a web browser.

The repository displays the initial commit message, README.md file, and the uploaded project structure.

This confirms that the local Flask Podman project was successfully version-controlled and published to GitHub for remote access and collaboration.

Conclusion:

To demonstrate the complete workflow of containerizing a Flask web application using Podman on Red Hat Enterprise Linux (RHEL) and managing it through Git and GitHub. The process began with configuring Git using the `git config` command to set up the username and email, followed by cloning the GitHub repository `assignment-rhel-podman` and creating an app directory to develop the Flask project. A Python file named `app.py` was then created using the `cat <<EOF` command, containing a simple Flask application that displays "Hello from Podman container on RHEL!" when accessed on port 8080. Next, a `requirements.txt` file listing Flask as the dependency and a `Dockerfile` specifying the container build instructions were created. The image was built using `podman build -t flask-podman-app .`, though it initially failed due to Docker Hub's unauthenticated pull rate limit, which was resolved by logging into Docker Hub using `podman login docker.io`. After successfully building the image, the Flask container was run using `podman run -p 8080:8080 flask-podman-app`, confirming the web app was accessible at `http://127.0.0.1:8080`. The user then staged and committed project files to Git and attempted to push them to GitHub, encountering authentication errors due to GitHub's new token-based security requirement. Finally, the uploaded repository `assignment-rhel-podman` was verified on GitHub, displaying the committed files and confirming successful version control integration. Overall, this experiment showcased the complete DevOps workflow — from application creation and containerization using Podman to source code management and cloud repository hosting using Git and GitHub on RHEL.